

Big Data

✍ *Dr. Soumaya EL MAMOUNE*
Soumaya.elmamoune@gmail.com

AVANT-PROPOS

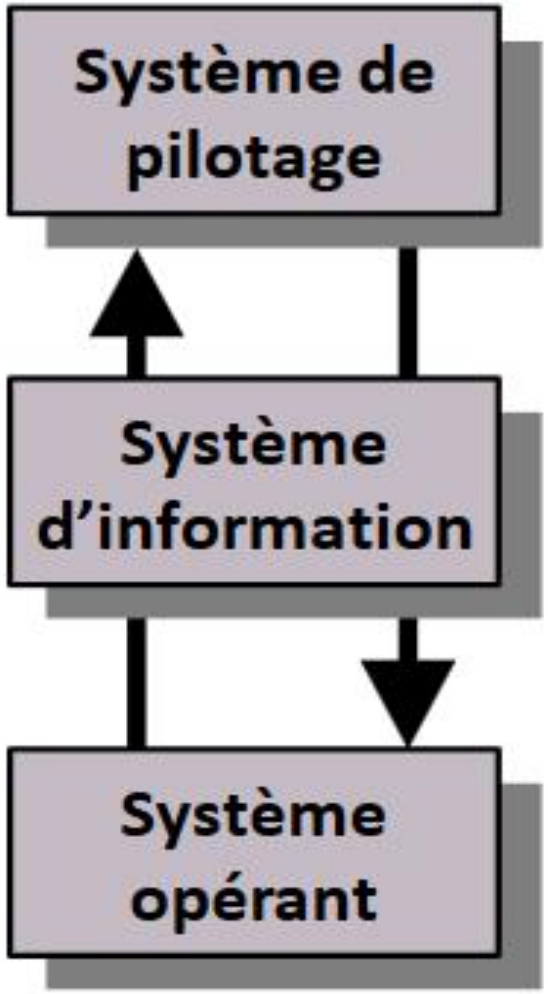
- Ce document est un support de cours qui se développe au fil des années afin de donner un support résumé et simplifié aux étudiants.
- Ce support de cours est développé selon un plan pédagogique qui a objectif de présenter un tutoriel simple et efficace. Afin de renforcer les connaissances, plusieurs exemples et exercices d 'application sont présentés vers la fin de ce documents.

Big Data?

Pourquoi utiliser le Big Data ?



CHAPITRE I- INTRODUCTION BIG DATA



Activité :
- réfléchir : adaptation à l'environnement, conception
- décider : prévisions, allocation, planification
- contrôler : qualité



Activité :
- générer des informations
- mémoriser
- diffuser
- traiter



Activité :
- transformer
- produire





Valable ?

TRACES NUMÉRIQUE



TRACES NUMÉRIQUE

- La notion de Big Data est un concept s'étant popularisé en 2012 pour traduire le fait que les entreprises sont confrontées à des volumes de données à traiter de plus en plus considérables et présentant un fort enjeux commercial et marketing
- Ces Grosses Données en deviennent difficiles à travailler avec des outils classiques de gestion de base de données
- Il s'agit donc d'un ensemble de technologies, d'architecture, d'outils et de procédures permettant à une organisation de très rapidement collecter, traiter et analyser de larges quantités et contenus hétérogènes et changeants et d'en extraire les informations pertinentes à un coût accessible.

TRACES NUMÉRIQUE

- L'utilisation des données numériques : ***collection***, ***stockage***, ***l'analyse***…
- Objectif : L'amélioration, la prise de décision, la prévision, la gestion des risques
- Cette utilisation devient plus en plus compliqué avec la variété et le volume de données circulées



LES DONNÉES NUMÉRIQUES

- L'explosion ***quantitative*** des données numériques a obligé les chercheurs à trouver de nouvelles manières de voir et d'analyser le monde des données.
- Il s'agit de découvrir de nouveaux ordres de grandeur pour capture, recherche, partage, stockage, analyse et présentation des données.
- le BD est partout, et utile pour l'entreprise : e-commerce, publicité, comportement de navigation sur Internet… Il s'agit d'un concept permettant de stocker un nombre ***indicible*** d'informations sur une base numérique.

LES DONNÉES NUMÉRIQUES

- Dans le domaine de l'usage, il répond à une nécessité de travailler la donnée plus profondément.
- Si les quantités de données semblent impressionnantes, encore faut-il savoir comment traiter et utiliser le big data pour les rendre utile à votre entreprise.

DÉFINITION DE BIG DATA

- Le Big Data (BD), littéralement « grosses données », ou méga données, parfois également appelés données massives. Il désigne un ensemble très volumineux de données qu'aucun outil classique de gestion de base de données ou de gestion de l'information ne peut vraiment travailler. Inventé par les géants du web, Le BD se présente comme une solution dessinée pour permettre à tout le monde d'accéder en temps réel à des bases de données géantes.

GÉNÉRALITÉS



GÉNÉRALITÉS

– Outils numériques plus performants et plus connectés: ordinateurs et smartphones





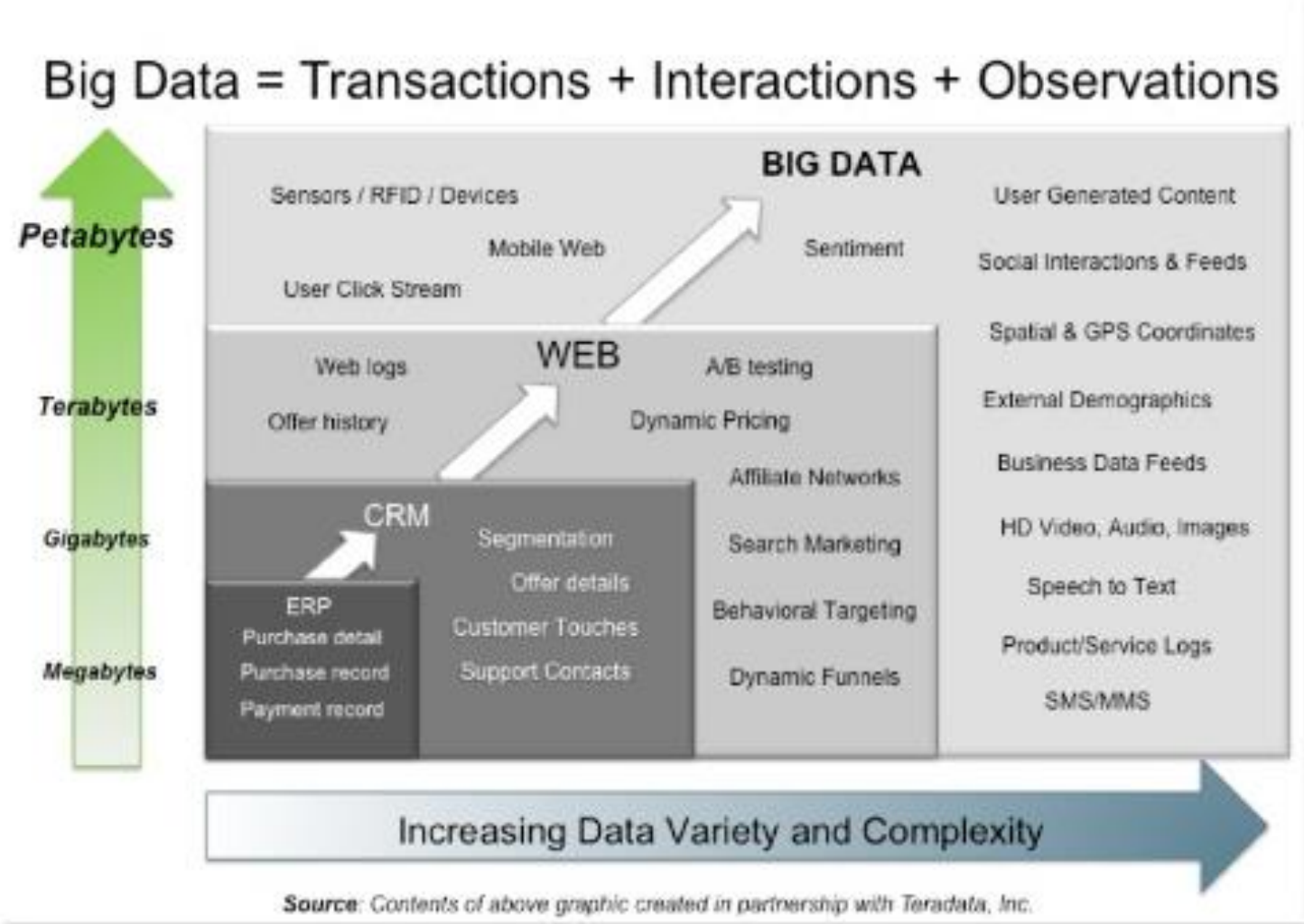
– Open Data
  - Réseaux sociaux: facebook
  - Données d'administrations publiques





– Internet des Objets
  - RFID (cartes de transport, codes bar, …)
  - Ericsson a estimé le nombre d'objets connectés dans le monde à 50 milliards en 2020 (12 milliards en 2013)

GÉNÉRALITÉS



Source: *Contents of above graphic created in partnership with Teradata, Inc.*

La croissance des données

- **Chaque jour, 2,5 trillions ($2.5*10^{18}$) d'octets de données sont générées.**
- **90% des données ont été créées au cours des deux dernières années**

2.5 million items per minute

300,000 tweets per minute

200 million emails per minute

220,000 photos per minute



2020

40 zettabytes



2012

2.8 zettabytes

80%

Of world's data is unstructured



Organizations need deeper insights

1 in 3 Business leaders frequently make decisions based on information they don't trust, or don't have

1 in 2 Business leaders say they don't have access to the information they need to do their jobs

83% of CIOs cited "Business intelligence and analytics" as part of their visionary plans to enhance competitiveness

60% of CEOs need to do a better job capturing and understanding information rapidly in order to make swift business decisions

Information is at the center of a new wave of opportunity

Système d'unités / Système binaire d'unités

International System of Units (SI)			Binary Usage (deprecated)
kilobyte	KB	10^3	2^{10}
megabyte	MB	10^6	2^{20}
gigabyte	GB	10^9	2^{30}
terabyte	TB	10^{12}	2^{40}
petabyte	PB	10^{15}	2^{50}
exabyte	EB	10^{18}	2^{60}
zettabyte	ZB	10^{21}	2^{70}
yottabyte	YB	10^{24}	2^{80}

International Electrotechnical Commission (IEC) - 1999		
kibibyte	KiB	2^{10}
mebibyte	MiB	2^{20}
gibibyte	GiB	2^{30}
tebibyte	TiB	2^{40}
pebibyte	PiB	2^{50}
exbibyte	EiB	2^{60}
zebibyte	ZiB	2^{70}
yobibyte	YiB	2^{80}

Source: Wikipedia, http://en.wikipedia.org/wiki/Kibibyte

Définition

- **Lisa Arthur (Forbes contributor)** définit la **Big Data:**

"*a collection of data from traditional and digital sources inside and outside a company that represent a source of ongoing discovery and analysis."*

- **Quelques exemples d'utilisation de la Big Data**

- Science
- Astronomy
- Atmospheric science
- Genomics
- Biogeochemical
- Biological
- Social
- Social networks
- Social data
  - * **Twitter**
  - * **Facebook**
  - * **LinkedIn**
- Commercial
- Web / event / database logs
- Sensor networks
- Internet text and documents
- **Medical** records
- Photographic archives
- Video / audio archives
- Government
- Military and homeland security surveillance

Exemples de volumétrie (2017)

- 3,3 millions de publications sur Facebook sont créées chaque minute.
- 65 972 photos Instagram sont téléversées chaque minute.
- 448 800 tweets sont crées chaque minute.
- 500 heures de vidéos YouTube sont téléversées chaque minute.
- On analyse moins de 1% des données créées .
- Il est impossible de suivre le rythme de croissance des données et il n'est pas nécessaire de tout stocker.
- En 2020, on estime à 37% les données utiles susceptibles d'être analysées

Données au repos et Données en mouvement

- Informations rapides en temps réel, souvent transitoires sur les réseaux:
  - Informations provenant de capteurs
  - Informations provenant des journaux en temps réel et des moniteurs d'activité
  - Contenu en streaming comme l'audio et la vidéo
  - Transactions à grande vitesse telles que les tickers (*rapports d'évolutions de prix*), systèmes de trafic, ...et



Information streams

- Informations stockées en dehors des systèmes conventionnels.

Les données peuvent provenir du Web ou de différents systèmes internes.

  - Collection de données qui étaient en mouvement
  - Informations provenant des médias sociaux, journaux, courriels, ...
  - Documents non structurés ou semi-structurées
  - Données structurées de divers systèmes



Information oceans

LES CARACTÉRISTIQUES DU BIG DATA

- 2001 : 3Vs (Gartener)



- 2012 : 4Vs (IBM)



LES CARACTÉRISTIQUES DU BIG DATA

- 2015 : 5Vs

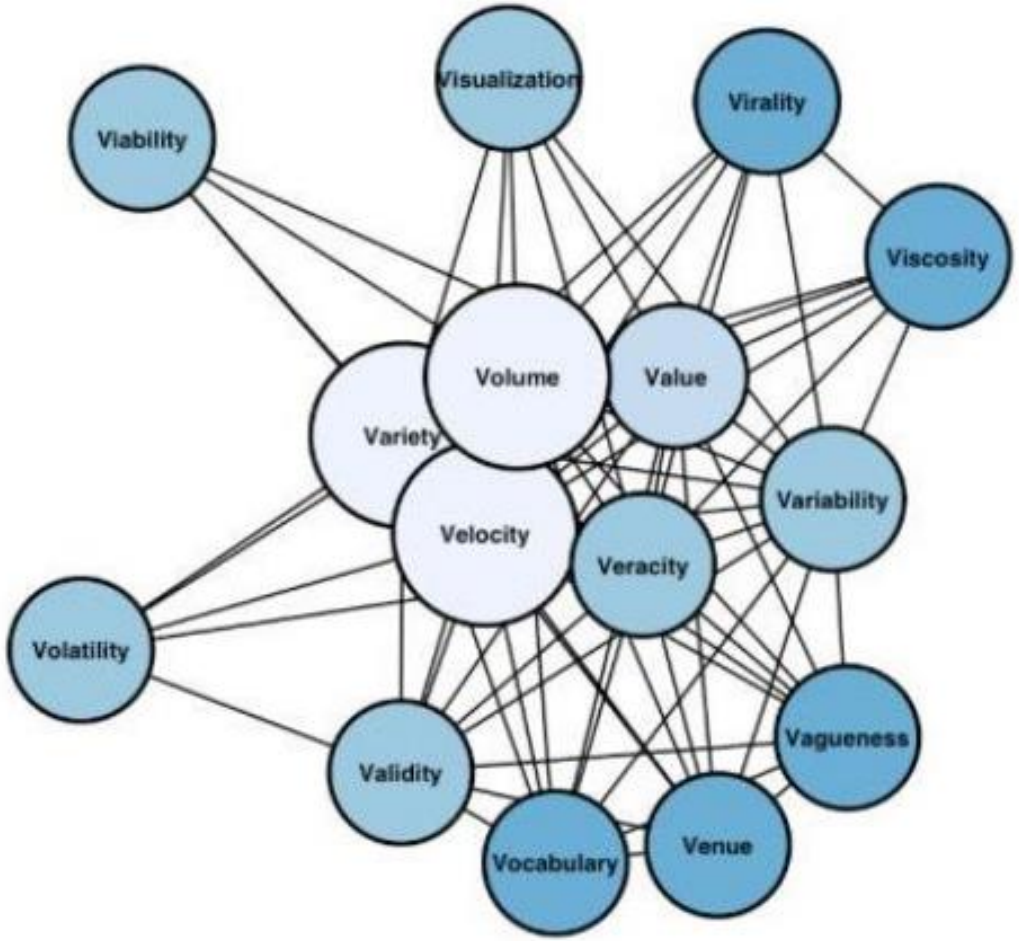


- 2016: : 7Vs



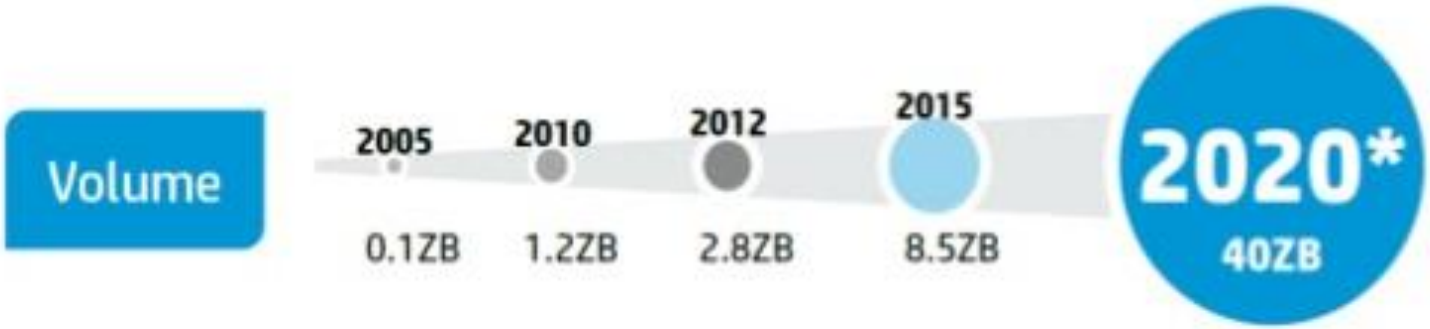
LES CARACTÉRISTIQUES DU BIG DATA

- 2017 : 10Vs ?



LES CARACTÉRISTIQUES DU BIG DATA

- **Volume (1/2)**
  - Croissance sans cesse des données à gérer de tout type, souvent en teraoctets voir en petaoctets.
  - Chaque jour, 2.5 trillions d'octets de données sont générées.
  - **90% des données créées dans le monde l'ont été au cours des 2 dernières années (2014).**
  - Prévision d'une croissance de 800% des quantités de données à traiter d'ici à 5 ans.



LES CARACTÉRISTIQUES DU BIG DATA

– **Volume (2/2)**
  - 4,4 zettaoctets de données
    = 4,4 trillion de gigaoctets
  - En 2013, il y a autant de données que les étoiles connues dans tout l'univers.

  - 44 zettaoctets de données
    = 44 milliards gigaoctets
  - 62 fois le nombre de tous les sables dans toutes les plages de la terre.

octets

1 Mégaoctet = 10^6 octets
1 Gigaoctet = 10^9 octets
1 Téraoctet = 10^{12} octets
1 Pétaoctet = 10^{15} octets
1 Exaoctet = 10^{18} octets
1 Zettaoctet = 10^{21} octets

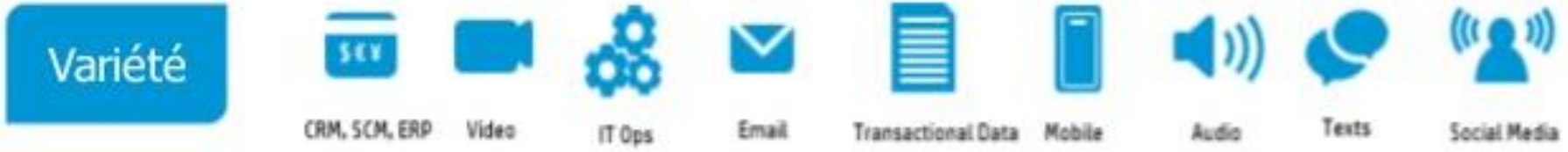
40
zettaoctets
de données
en 2020

50
milliards
d'objets connectés
à la même date

LES CARACTÉRISTIQUES DU BIG DATA

– **Variété**

- Traitement des données sous forme structurée (bases de données structurée, feuilles de calcul venant de tableur, …) et non structurée (textes, sons, images, vidéos, données de capteurs, fichiers journaux, medias sociaux, signaux,…) qui doivent faire l'objet d'une analyse collective.



- Diversité des données



LES CARACTÉRISTIQUES DU BIG DATA

– **Vitesse (Velocity)**
  - Utilisation des données en temps réel (pour la détection de fraudes, analyse des données, …).
  - Fait référence à la vitesse à laquelle de nouvelles données sont générées et la vitesse à laquelle les données sont traitées par le système pour être bien analysées.
  - La technologie nous permet maintenant d'analyser les données pendant qu'elles sont générées, sans jamais mettre en bases de données.

  - Streaming Data
    - ➢ des centaines par seconde

  - 100 Capteurs
    - ➢ dans chaque voiture moderne pour la surveillance



LES CARACTÉRISTIQUES DU BIG DATA

- **Véracité**
  - Fait référence à la qualité de la fiabilité et la confiance des données.
  - Données bruités, imprécises, prédictives, …

  - La génération des données par Spambots est un exemple digne de confiance.
    - ➢ L'élection présidentielle de 2012 au Mexique avec de faux comptes Twitter.

  - DES MILLIONS DE DOLLARS $ PAR AN
    - ➢ Ce que la pauvre qualité des données coute pour l'économie des Etats-Unis.

  - 1 à 3 CHEFS D'ENTREPRISE
    - ➢ Ne font pas confiance à l'information qu'ils utilisent.

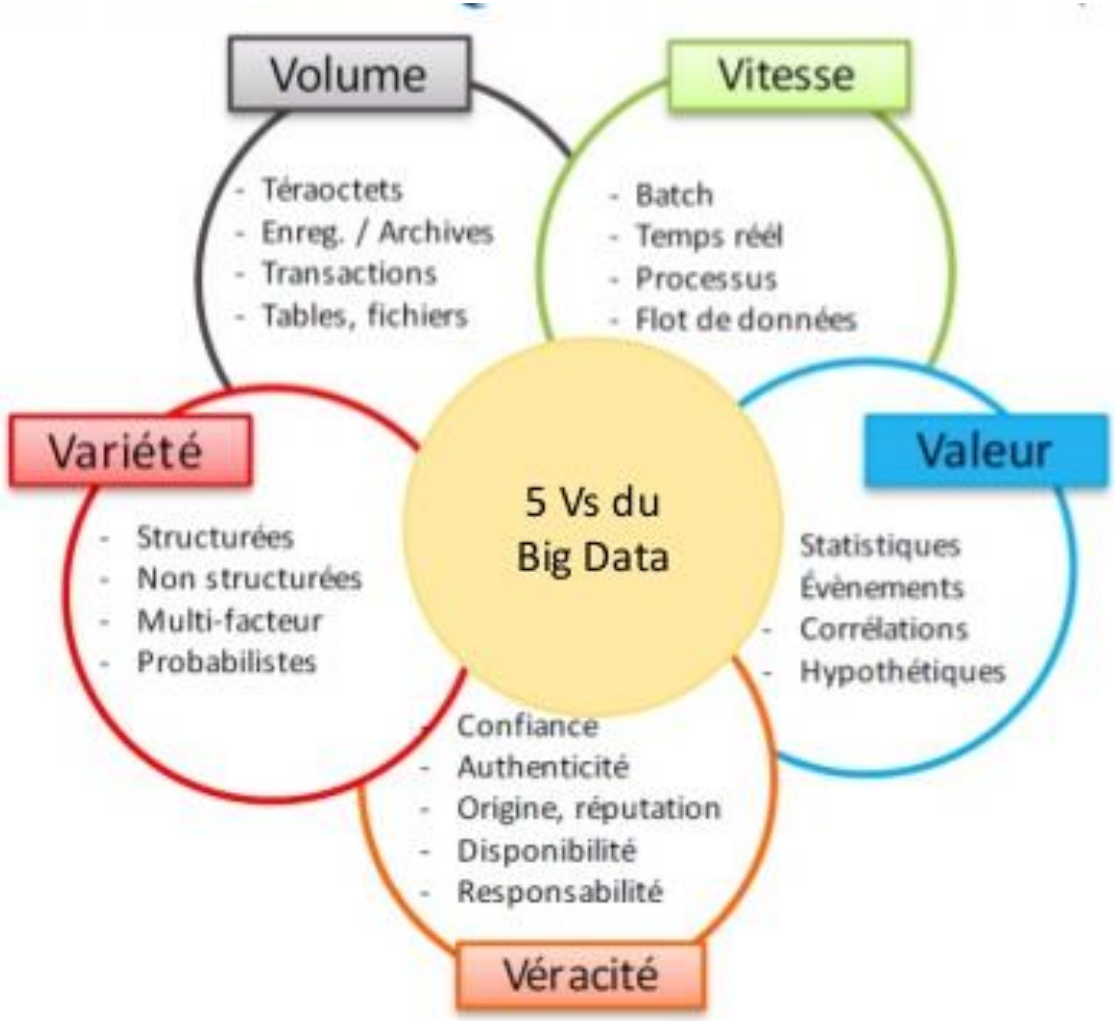


LES CARACTÉRISTIQUES DU BIG DATA

– **Valeur**
  - La démarche Big Data n'a de sens que pour atteindre des objectifs stratégiques de **création de valeur** pour les clients et pour l'entreprise; dans tous les domaines d'activité : commerce, industrie, services …
  - Le succès d'un projet Big Data n'a d'intérêt aux utilisateurs que s'il **apporte de la valeur ajoutée** et de nouvelles connaissances.



LES CARACTÉRISTIQUES DU BIG DATA



Caractéristiques du Big Data

- Il n'y a pas de définition unique du Big Data, mais certains éléments sont communs à toutes les définitions: la **vélocité**, le **volume**, la **variété**.
- Extraire, de manière rapide et rentable, des connaissances à partir de données volumineuses , variées et d'une vitesse élevée.



Volume: varie de terabytes à zettabytes. On doit traiter efficacement ce volume croissant.

Variété: Gérer divers types et structures de données

Vélocité: Analyser les flux de données en continu et les grands volumes de données persistantes.

Véracité: authenticité et fiabilité des données necessitant une *grande rigueur* dans la collecte, le recoupement, le croisement et l'enrichissement des données pour lever l'incertitude pour garantir l'intégrité des données.

Valeur: le point le plus important, une solution Big Data doit apporter une valeur ajoutée pour l'entreprise en répondant à des objectifs commerciaux ou Marketing qui orientent l'utilisation des Big Data.

LES AVANTAGES DU BIG DATA

- Aide intelligente pour la prise de décision,
- Réductions des coûts
- Réductions de temps
- Optimisation des processus de l'entreprise et développement de nouveaux produits ou services

- ….

Big Data dans un système décisionnel

Traditional Approach
Structured & Repeatable Analysis

Business Users
Determine what question to ask



IT
Structures the data to answer that question



Monthly sales reports
Profitability analysis
Customer surveys

Big Data Approach
Iterative & Exploratory Analysis



IT
Delivers a platform to enable creative discovery



Business
Explores what questions could be asked

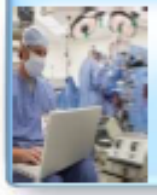
Brand sentiment
Product strategy
Maximum asset utilization

Secteurs d'utilisation des Big Data





Analyse du sentiment et de l'expérience client multicanal, ...



Détection à temps des causes potentiellement mortelles dans les hôpitaux pour intervenir, ...



Prédire les conditions météorologiques pour planifier l'utilisation optimale des éoliennes et optimiser les dépenses , ...



Prendre des décisions de risque basées sur des données transactionnelles en temps réel, ...



Identifier les criminels et les menaces provenant de sources vidéo, audio et de données disparates

Projets Big Data

- **Déforestation** : Projet Planetary SKIN (Par la NASA, CISCO, …)
- **Suivi astronomique en direct** : Projet LSST (Large Synoptic Survey Telescope) (20 Tb chaque nuit)
- **Traitement du Cancer**: projet ICGC (International Cancer Genome Consortium), analyse de plusieurs BD sur des tumeurs de 5O types de cancers
- **Détection d'épidémies en temps réel** : Projet Healthmap puis ResistanceOpen pour la collecte, le stockage et l'analyse de divers types de données pour le suivi et la surveillance d'épidémie au niveau mondial.

Approche traditionnelle Vs Approche Big Data

Approche traditionnelles

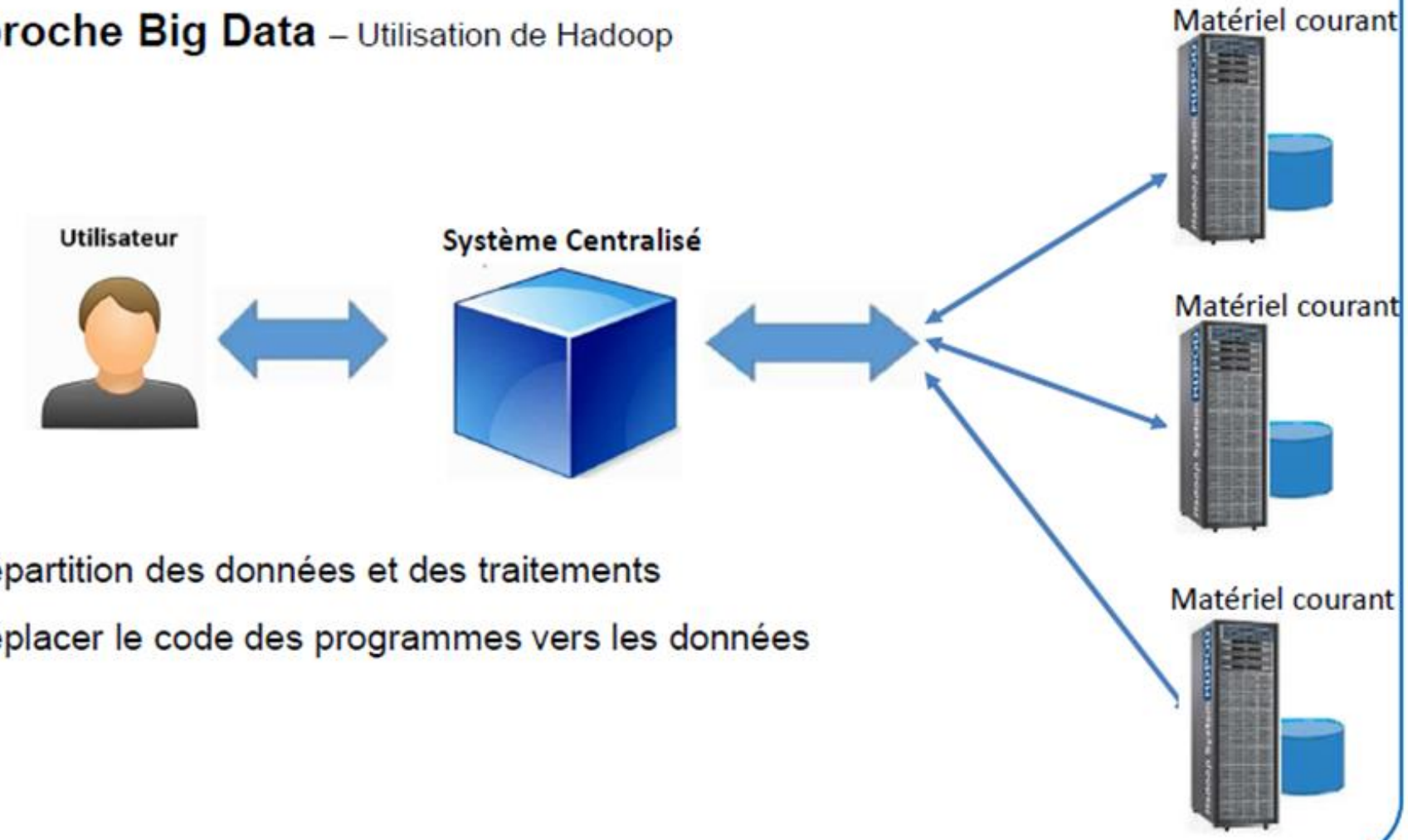


Insuffisance lorsqu'il y a:

- Grande croissance de données
- Des données sans schémas

Approche traditionnelle Vs Approche Big Data

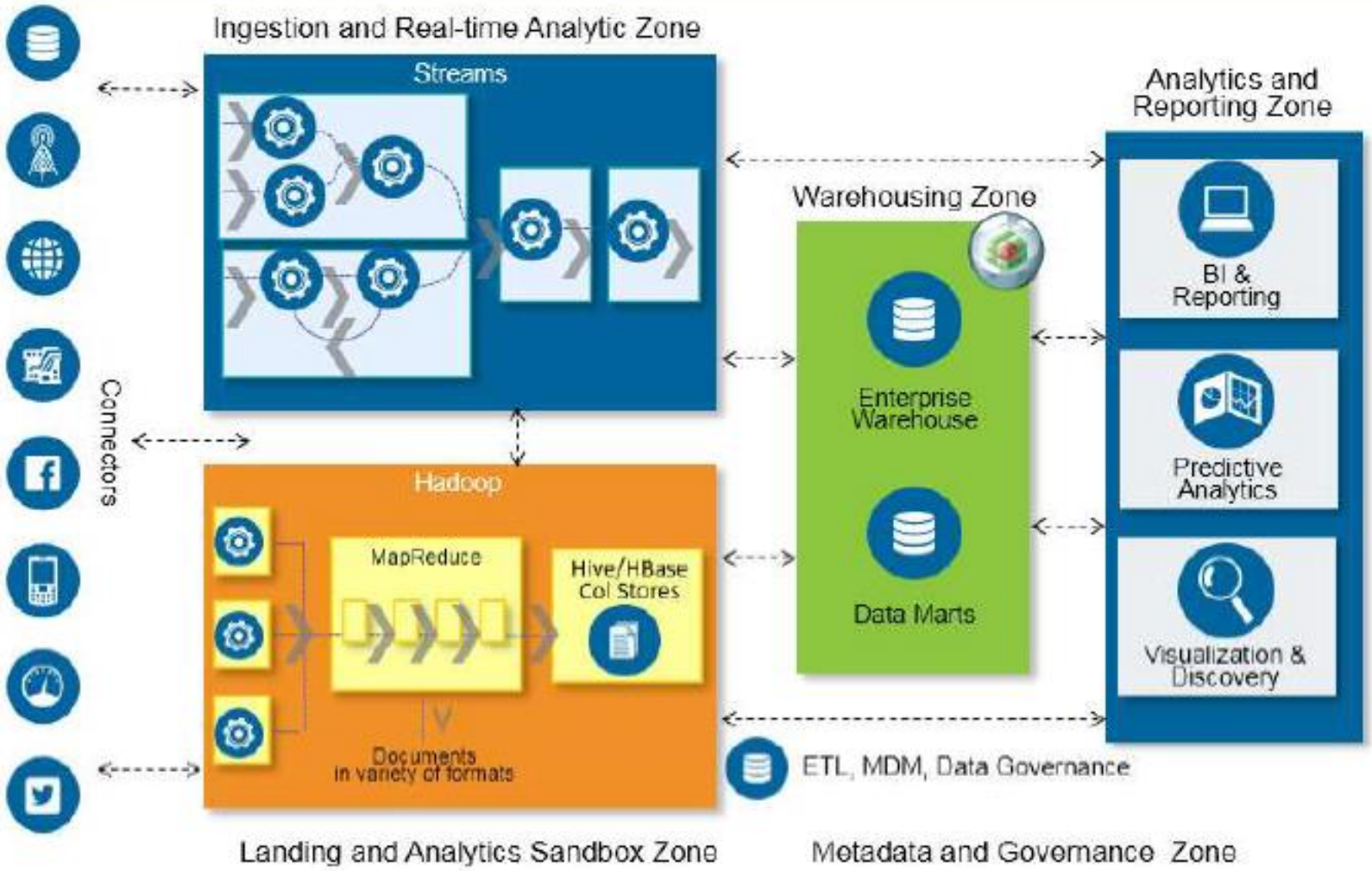
Approche Big Data – Utilisation de Hadoop



- Répartition des données et des traitements
- Déplacer le code des programmes vers les données

CHAPITRE 2- HDFS

Intégration Hadoop / S.I et Décisionnel

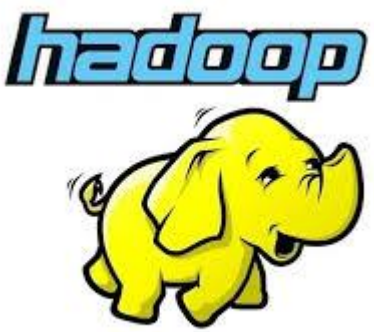


Introduction

- La vitesse de lecture et d'écriture (*Latence du disque*) ne s'est pas beaucoup amélioré au cours des dernières années (*autour de 70 - 80MB / sec – disque récents* jusqu'à 320 MB/s (selon l'interface ATA/IDE ou SCSI – SSD peut atteindre 1000M/s)
- **Problème:** Combien de temps faut-il pour lire 1 To de données?

  1 To (à 80 Mo / sec):
  - 1 disque - 3,4 heures
  - 10 disques - 20 min
  - 100 disques - 2 min
  - 1000 disques - 12 sec
- **Solution:** Le traitement parallèle des données
  - **Calcul distribué:** plusieurs ordinateurs apparaissent comme un super ordinateur, communiquent les uns avec les autres par le passage de messages, opèrent ensemble pour atteindre un but commun.
- **Défis:** Hétérogénéité (matériel-OS) - Ouverture - Évolutivité - Sécurité - Concurrence - Tolérance aux pannes - Transparence

HADOOP



- Hadoop est un framework logiciel open source permettant de stocker des données, et de lancer ds applications sur des grappes de machines standards. Cette solution offre un espace de stockage massif pour tous les types de données, une immense puissance de traitement et la possibilité de prendre en charge une quantité de tâches virtuellement illimitée. Basé sur Java, ce framework fait partie du projet Apache, sponsorisé par Apache Software Foundation.

POURQUOI

Pourquoi Hadoop est important ?

- Les avantages apportés aux entreprises par Hadoop sont nombreux. Grâce à ce framework, il est possible de stocker et de traiter de vastes quantités de données rapidement. Face à l'augmentation en hausse du volume de données et à leur diversification, principalement liée aux réseaux sociaux et à l'internet des objets, il s'agit d'un avantage non négligeable.
- le modèle de calcul distribué d'Hadoop permet de traiter rapidement le Big Data. **Plus le nombre de nœuds de calcul utilisés est important, plus la puissance de traitement est élevée.** Les données et les applications traitées sont protégées contre les échecs hardware. Si un nœud tombe en panne, les tâches sont directement redirigées vers d'autres nœuds pour s'assurer que le calcul distribué n'échoue pas. De multiples copies de toutes les données sont stockées automatiquement.

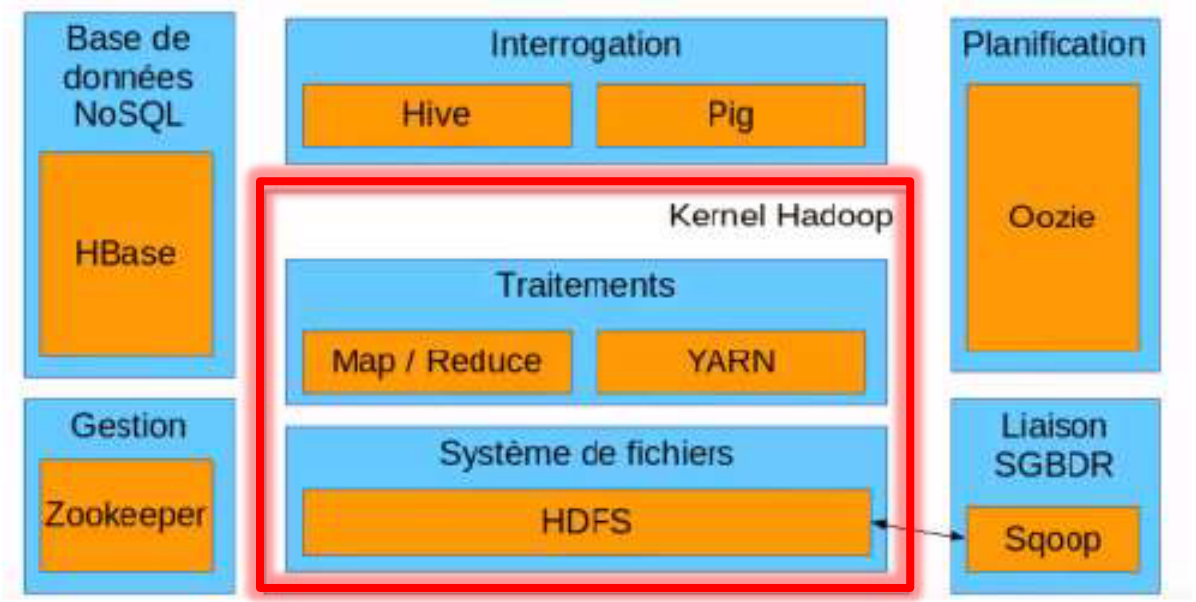
POURQUOI

Pourquoi Hadoop est important ?

- Le framework open source est donc **gratuit et repose sur des machines standards pour stocker de larges quantités de données**. Enfin, il est possible d'adapter le système pour prendre en charge plus de données en ajoutant simplement des nœuds. L'administration requise est minimale.

L'écosystème Hadoop

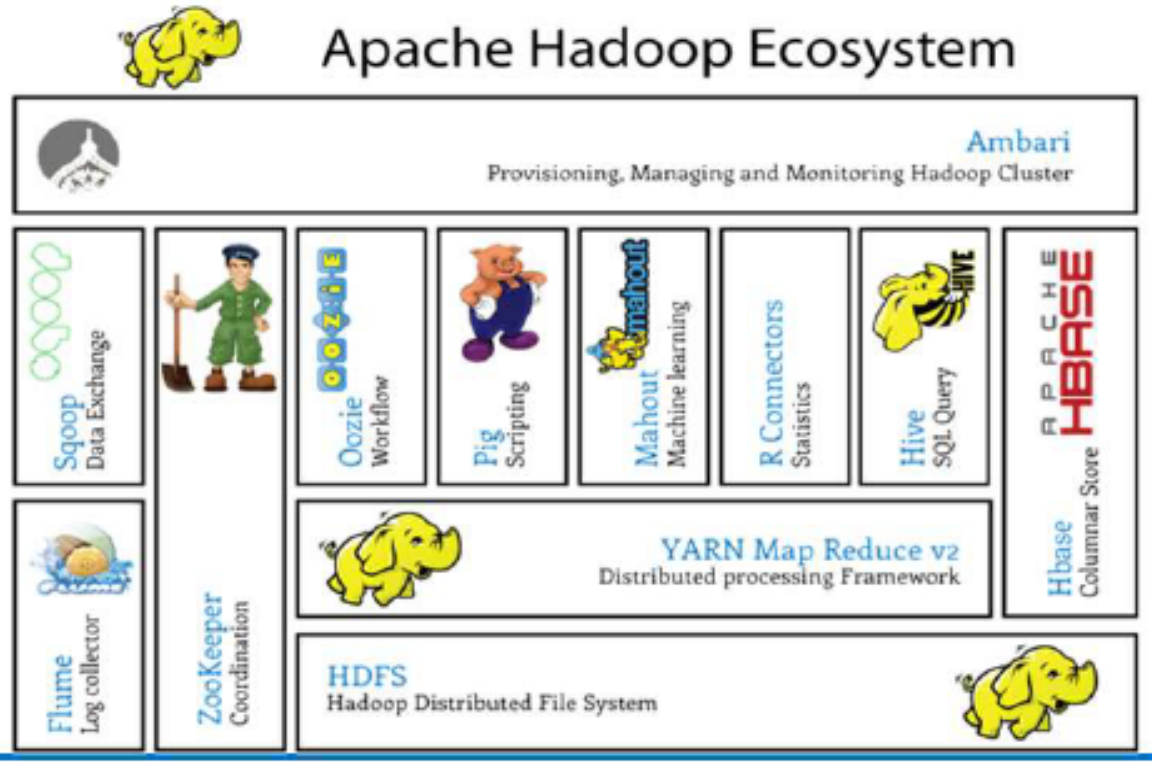
- Hadoop est basé sur:
  - **MapReduce/YARN**: framework de traitement distribué
  - **HDFS** : pour un stockage distribué
- Hadoop est écrit en **Java** et sous License Apache
- Plusieurs projets sont liés à Hadoop ⇨ Ecosystème Hadoop



Simon Gilliot : Introduction à la plate-forme Hadoop et à son écosystème

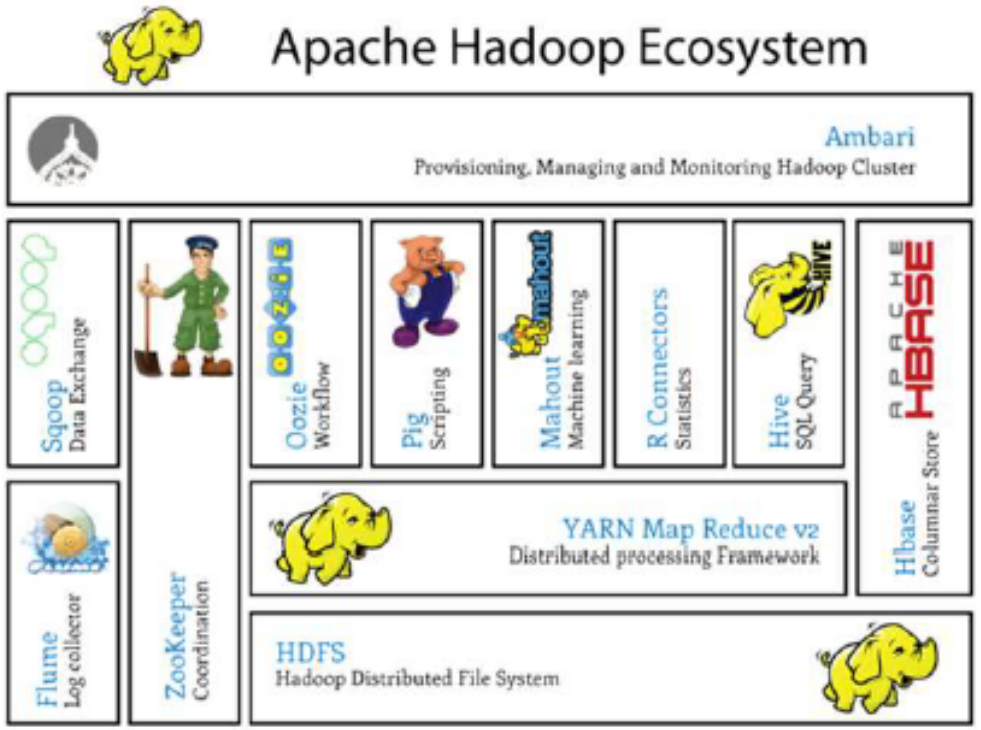
L'écosystème Hadoop

- Composants de base: **HDFS – MapReduce /Yarn**
- D'autres outils existant pour:
  - La simplification des opérations de traitement sur les données
  - La gestion et coordination de la plateforme
  - Le monitoring du cluster



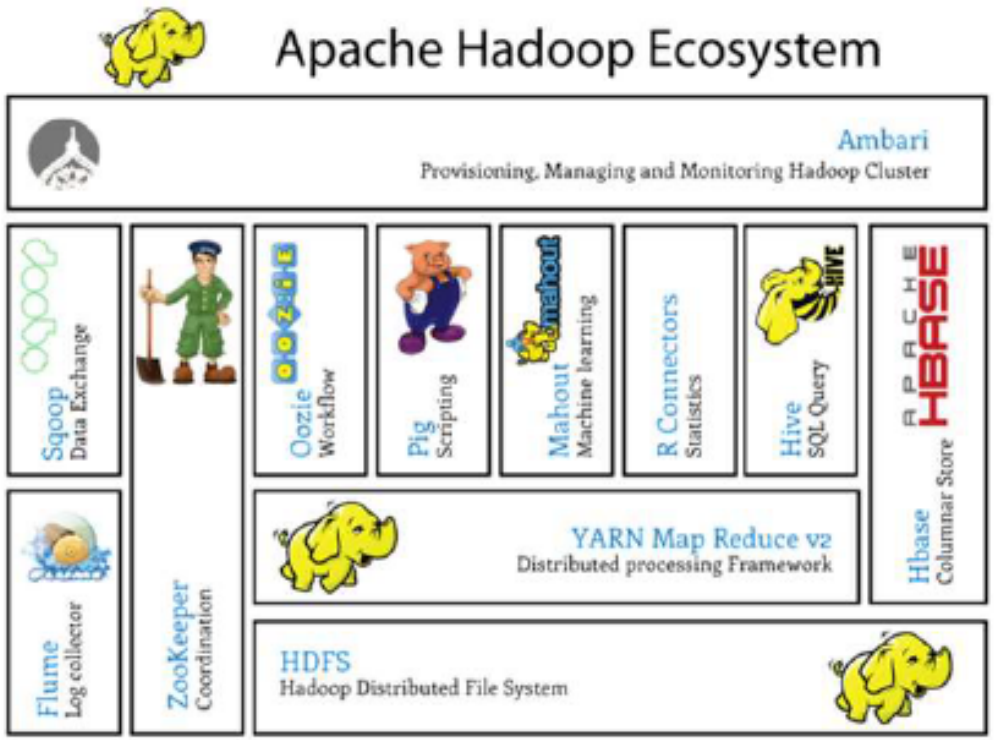
L'écosystème Hadoop

- **Pig**: Langage de script
- **Hive**: Langage proche de SQL (Hive QL)
- **R Connectors**: permet l'accès à HDFS et l'exécution de requêtes Map/Reduce à partir du langage R
- **Mahout**: bibliothèque de machine learning et mathématiques
- **Oozie**: permet d'ordonnancer les jobs Map Reduce (Java, Python, Pig, Hive, …), en définissant des workflows



L'écosystème Hadoop

- **Hbase** : Base de données NoSQL orientée colonnes.
- **Sqoop**: Lecture et écriture des données à partir de BD externes
- **Flume**: Collecte de logs et stockage dans HDFS
- **Ambari**: outil pour la gestion et monitoring des clusters
- **Zookeeper**: fournit un service centralisé pour maintenir les information de configuration, de nommage et de synchronisation distribuée



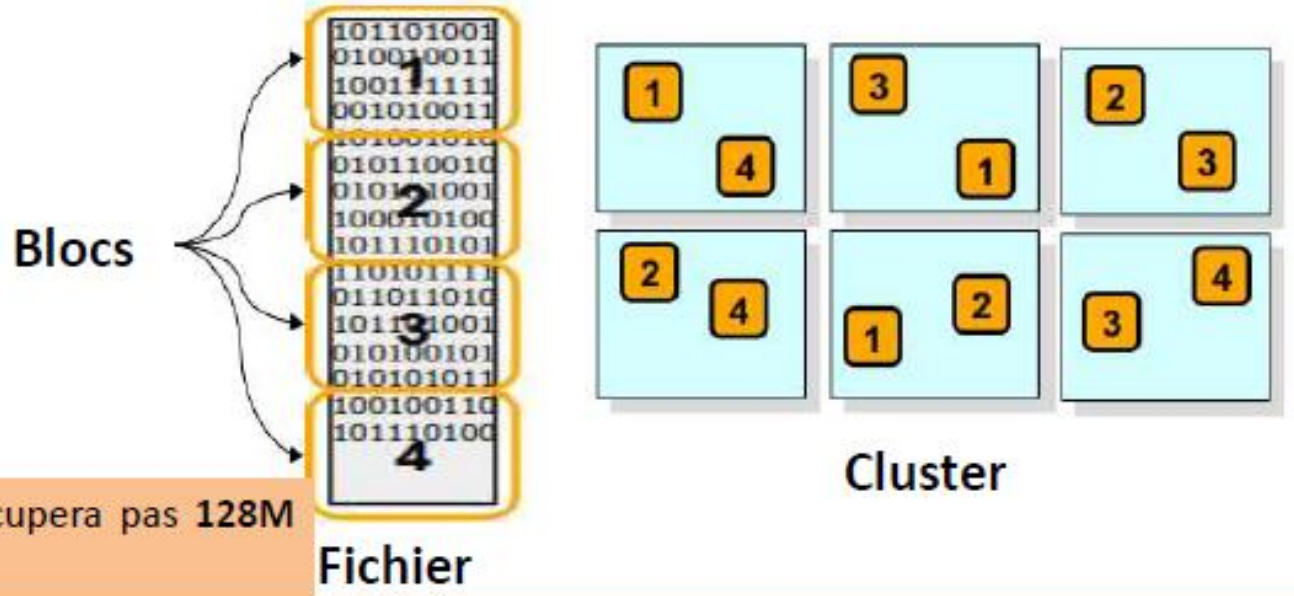
Hadoop: HDFS

HDFS : Système de fichiers conçu pour stocker des fichiers très volumineux dans un cluster d'ordinateurs pas chers.

- HDFS stocke les données sur plusieurs nœuds (**cluster**)
- Il existe des clusters **Hadoop** qui stockent des Po de données.
- Ecrire une fois/lire plusieurs fois (***WORM***: *write once read many times*): Les données sont généralement générées ou copiées à partir de la source. Des analyses (batch) sont ensuite effectuées sur cet ensemble de données au fil du temps. Chaque analyse impliquera une grande proportion, sinon la totalité, de l'ensemble de données.
- HDFS réplique les données sur plusieurs nœuds afin d'éviter leur perte en cas de panne de certains nœuds ⇨ Fiabilité, Disponibilité
- Chaque fichier est décomposé en plusieurs blocs qui seront répliqués sur les nœuds.

Hadoop: HDFS

- **Bloc**: taille par défaut 128Mo sur Hadoop 2.x (64Mo sur Hadoop 1.x) (bloc OS = 4k Octets par exemple)
- La taille est grande pour minimiser le temps de recherche (localisation) des blocs.
- Chaque bloc est répliqué sur un petit nombre (***facteur de réplication***) de machines physiquement séparées. (*par défaut trois*)



Ce bloc n'occupera pas **128M** sur disque.

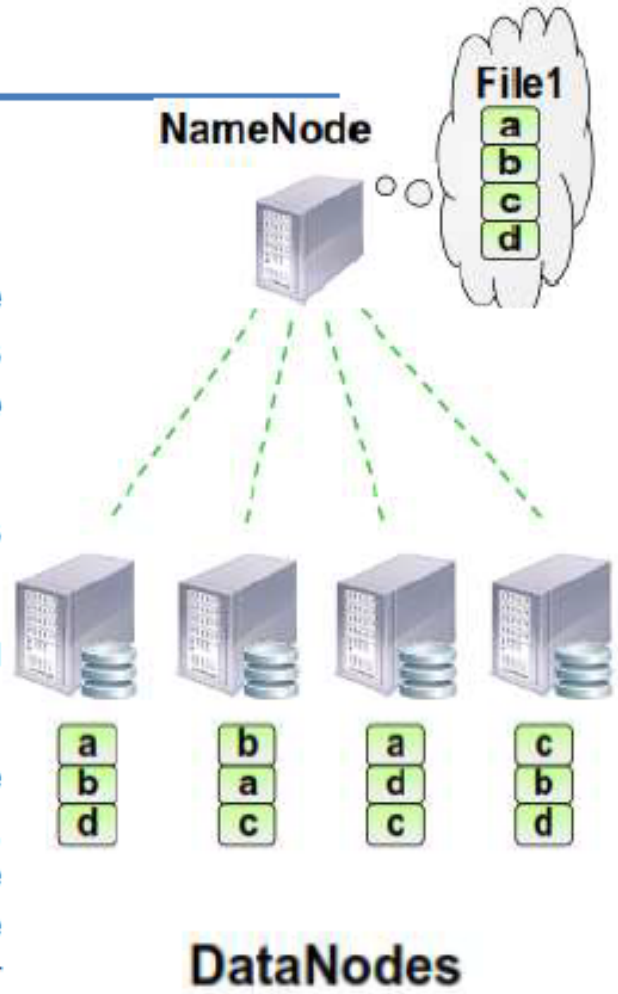
Hadoop: Architecture de HDFS

- Architecture **maitre/esclave**
- **NameNode**
  - Gère **l'espace de noms du système de fichiers**: Il gère l'arborescence du système de fichiers et les métadonnées de tous les fichiers (*noms, facteurs de réplication, propriétés*) et répertoires de l'arborescence.

Ces informations sont stockées sur le disque local dans deux fichiers:

- **FsImage** (*métadonnées*) chargé en mémoire au démarrage du **NameNode**.
- **EditLog** : pour enregistrer chaque modification apportée aux métadonnées du système de fichiers. Par exemple, création d'un nouveau fichier dans HDFS, le NameNode insère un enregistrement dans le **EditLog** indiquant cela. De même, la modification du facteur de réplication d'un fichier entraîne l'insertion d'un nouvel enregistrement dans **EditLog**

- Régule l'accès aux fichiers par les clients (*équilibrer la charge*)



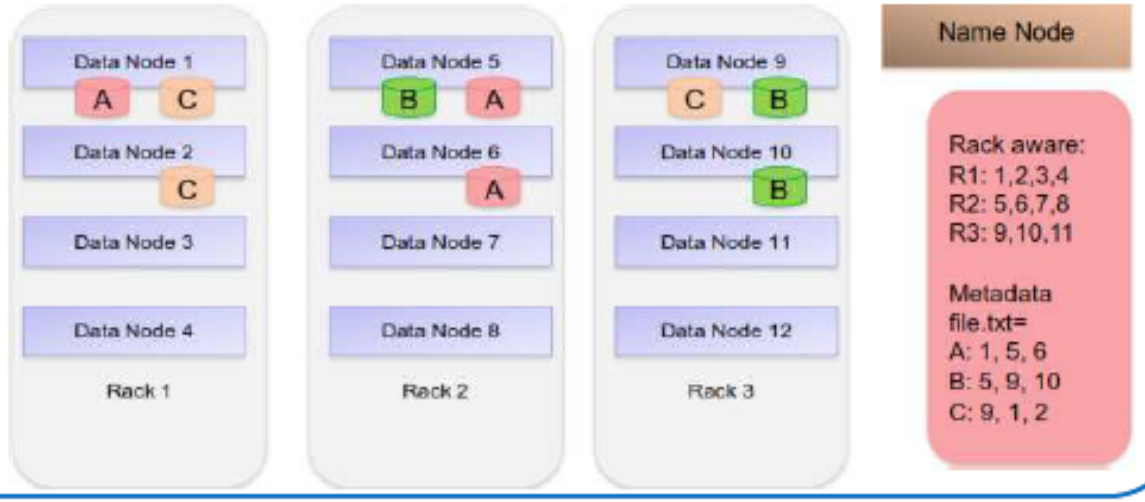
Réf: IBM BigInsights Foundation v4.0

(Course code DW613 ERC 1.0)

Hadoop: Architecture de HDFS

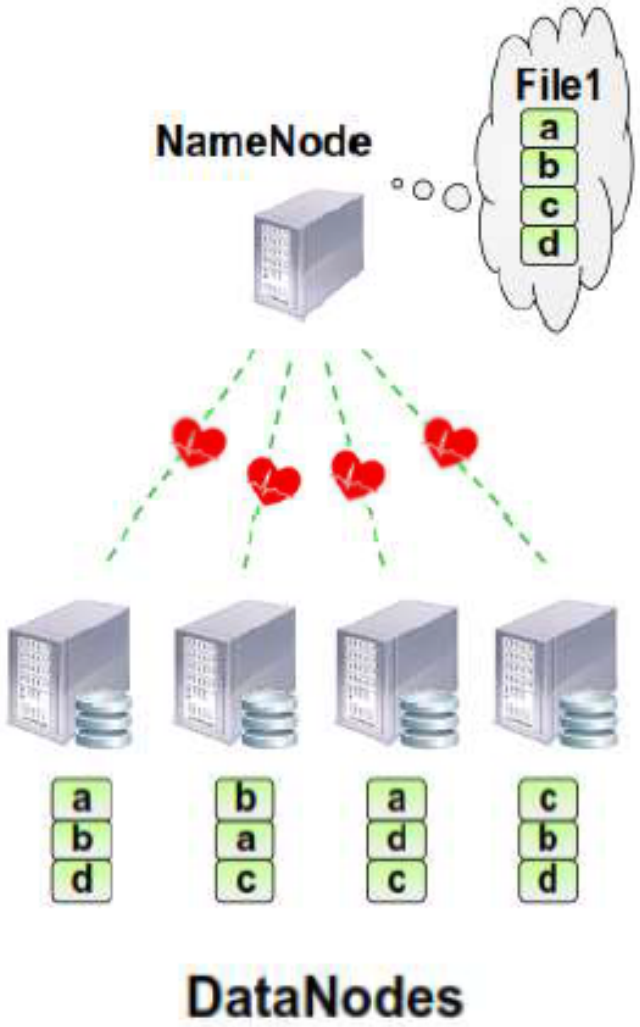
- Le **NameNode** conserve les **métadonnées** en **mémoire** pour assurer un accès rapide ce qui nécessite une grande taille mémoire pour son fonctionnement.
- Les blocs de fichiers sont répliqués sur plusieurs nœuds (***DataNodes***).
- Le nombre de copies de chaque bloc est contrôlé par le paramètre de réplication (Par défaut 3)
- Réplication sur plusieurs racks:

Une copie sur un premier rack - La 2ème et la 3ème copies peuvent être ensemble sur un autre Rack.



Hadoop: Architecture de HDFS

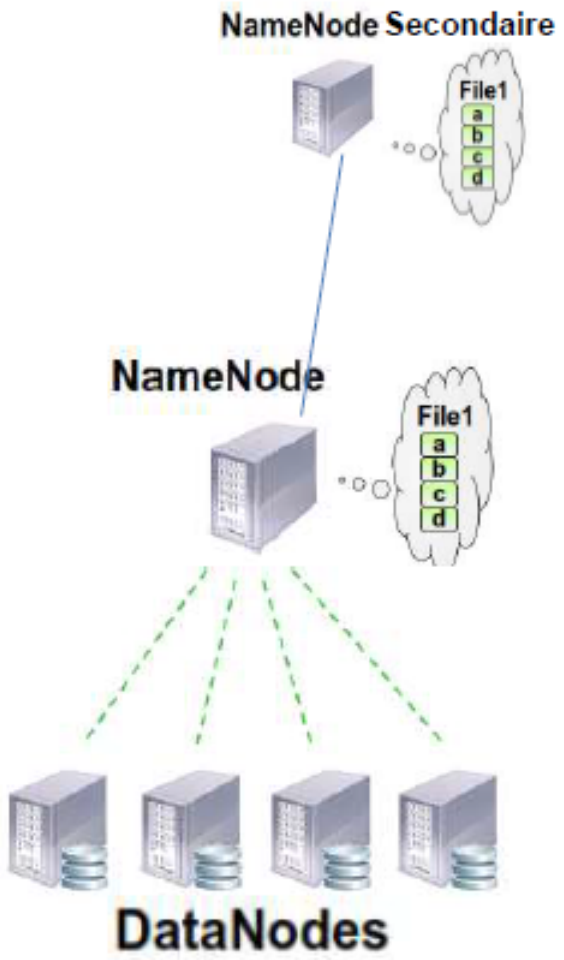
- **DataNode**
  - Nombreux DataNodes par cluster (***un par nœud***)
  - Gère le stockage (read/write) attaché aux nœuds
  - Envoie périodiquement un signal au **NameNode** sous forme de **heartbeat** (id, espace stockage, espace utilisé, …) avec un intervalle indiqué dans **hdfs-site.xml** (**dfs.heartbeat.interval** , par défaut 3s).
  - Renvoie périodiquement (**dfs.blockreport.intervalMsec** par défaut 21600000 Ms) au NameNode un rapport sur les blocs qu'il stocke (id, taille, date de création, …)
  - Si un **DataNode** tombe en panne, ses blocs seront répliqués sur d'autres **DataNodes** (**timout** = 2 * heartbeat.recheck.interval + 10 * heartbeat.interval)
  - Effectue également la création, la suppression et la réplication de blocs sur ordre du NameNode.
  - Les données sont stockées sur plusieurs nœuds



Hadoop: Architecture de HDFS

• Secondary NameNode

- Le NameNode est un point de défaillance unique (***Single Point of Failure - SPOF***): Si la machine exécutant le NameNode est perdue, toutes les informations (meta-data) sur les fichiers de données seraient perdues. Par conséquent, on ne pourra pas reconstruire les fichiers à partir des blocs sur les DataNodes.
- Hadoop sauvegarde les métadonnées du système de fichiers (**FsImage**): **NameNode** écrit son état persistant sur plusieurs systèmes de fichiers (disque local, disque distant).
- Utiliser un NameNode secondaire: n'a pas le même rôle que le NameNode. Il fusionne périodiquement l'image de l'espaces des noms (FsImage) et le journal d'édition (EditLog) pour éviter que ce dernier devienne trop volumineux. Il s'exécute sur une machine physique distincte (*nécessite beaucoup de CPU et de RAM pour effectuer la fusion*)



Hadoop: Architecture de HDFS

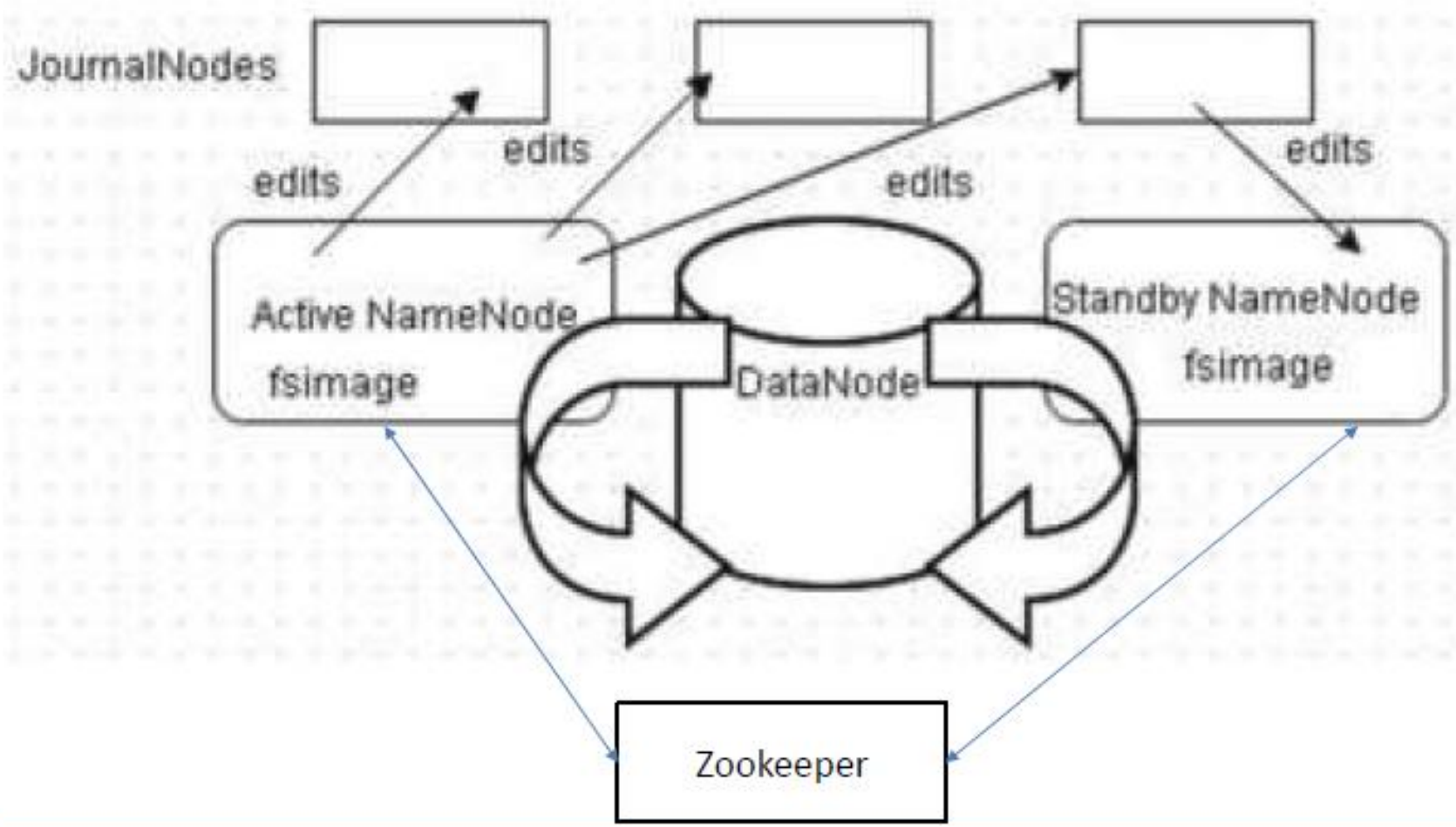
- **Secondary NameNode** *(suite)*
- La **réplication** des métadonnées du **NameNode** et l'utilisation du **NameNode secondaire** permet d'éviter la perte de métadonnées et de conserver des copies récentes.
- Mais elle n'offre pas une **haute disponibilité** du système de fichiers:
  - En cas d'échec du **NameNode**, tous les clients, y compris les jobs MapReduce, ne pourraient pas lire, écrire ou répertorier les fichiers, car le **NameNode** est le seul référentiel des métadonnées et du mapping fichier-bloc.
  - Dans un tel cas, l'ensemble du système Hadoop serait **hors service** jusqu'à ce qu'un nouveau **NameNode** puisse être mis en ligne.
- Pour récupérer un **NameNode défaillant**, l'administrateur démarre un **nouveau NameNode** avec l'une des copies de métadonnées du système de fichiers et configure les **DataNodes** et les applications clientes (du Data Center) pour utiliser ce nouveau **NameNode**.

Hadoop: Architecture de HDFS

- **Secondary NameNode** *(suite)*
- Le nouveau **NameNode** ne peut répondre aux demandes des clients tant qu'il n'a pas **(1)** chargé son **FsImage dans la mémoire**, **(2)** relu son journal d'édition (EditLog) , et **(3)** reçu suffisamment de rapports de blocs des **DataNodes** pour quitter le mode sans échec (Safe Mode).
- Sur les grands clusters avec un grand nombre de fichiers et de blocs, le temps nécessaire pour qu'un **NameNode** démarre à froid peut être de 30 minutes ou plus.
- Le long temps de récupération est également un problème pour l'entretien de routine.
- Hadoop 2 a remédié à cette situation en ajoutant la prise en charge de la haute disponibilité (**HA**) HDFS: il y a un pair de NameNodes Passif/Actif. En cas de panne du Namenode actif, le passif devient actif pour continuer à répondre aux demandes des clients sans interruption significative.

Hadoop: Architecture de HDFS

Haute Disponibilité dans HDFS (high-availability – HA)



Hadoop: Architecture de HDFS

Haute Disponibilité dans HDFS (high-availability – HA) (Hadoop 2)

1. Le NameNode actif et le NameNode passif communiquent avec un groupe (quorum) de JournalNodes (généralement 3)
2. Le NameNode actif envoie les "edits" (logs) aux JounalNodes pour les partager avec le NameNode passif.
3. Le NameNode passif récupère les "edits" depuis l'un des JournalNodes et les applique à la copie Fsimage qu'il a afin de la synchroniser avec celle du NameNode actif.
4. Les DataNodes envoient périodiquement les rapports sur les blocs aux NameNodes actif et passif.
5. ZooKeeper est un démon de coordination utilisé pour coordonner le basculement rapide du NameNode actif vers le NameNode passif. ZooKeeper a deux fonctions dans un cluster HDFS haute disponibilité:
   - Détecter une défaillance du NameNode actif. Les deux NameNodes conservent une session active dans ZooKeeper. Lorsqu'il y a défaillance du NameNode actif, ZooKeeper détecte cette défaillance suite à l'interruption de la session du NameNode actif.
   - ZooKeeper initie un basculement vers NameNode passif et fournit un mécanisme pour la l'élection du NameNode actif lorsqu'il y a plusieurs candidats (NameNodes passfis)

Hadoop: Modes de fonctionnement

Hadoop possède trois modes d'exécution :

- **Mode local (standalone)** : Hadoop fonctionne sur une seule machine et tout s'exécute dans la même JVM (*Java Virtual Machine*). En mode local le système de gestion de fichiers utilisé est celui du système hôte (Exemple Linux: ext3, ext4 ou xfs) et non HDFS.
- **Mode pseudo-distribué**: Hadoop fonctionne sur une seule machine, mais chacun des deamons principaux s'exécute dans sa propre JVM. Le système de fichier utilisé est HDFS dans ce mode.
- **Mode totalement distribué**: c'est le mode d'exécution réel d'Hadoop. Il permet de faire fonctionner le système de fichiers distribué et les deamons sur un ensemble de machines différentes.

Hadoop: Commandes HDFS

- Nous utiliserons Hadoop avec un seul nœud pour tester et utiliser quelques commandes HDFS.
- Les clients HDFS utilisent la propriété **fs.default.name** ou **fs.defaultFS** de Hadoop pour déterminer l'URL de l'hôte et le port du **NameNode** HDFS.
- Nous l'utiliserons sur localhost (**fs.defaultFS** = *hdfs://localhost:* **8020**) avec le port HDFS par défaut, **8020**.
- Les propriétés et paramètres Hadoop sont dans des fichiers de configuration:

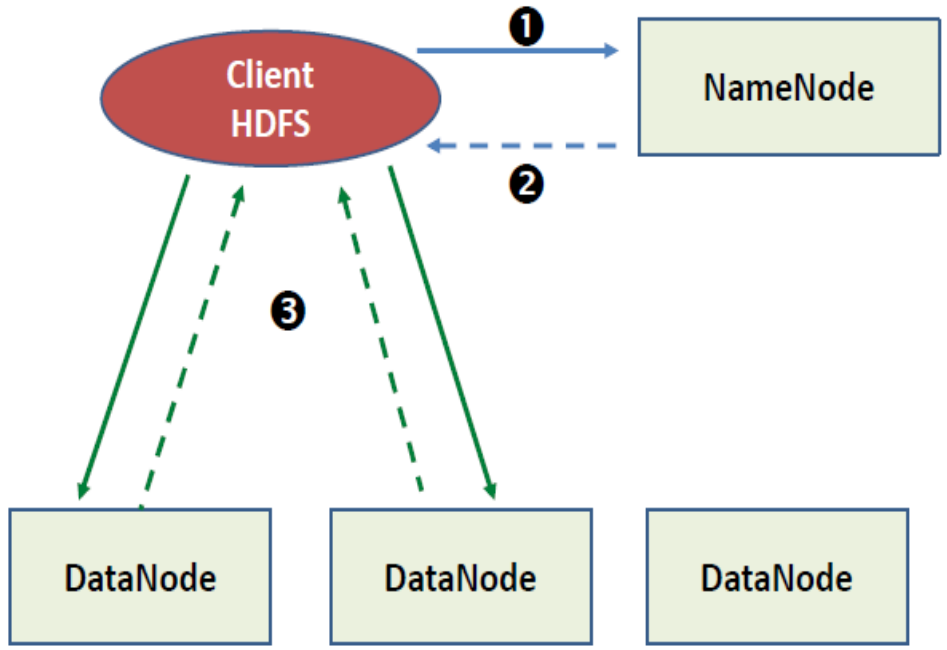
/etc/hadoop/conf/**core-site.xml**
/etc/hadoop/conf/**hdfs-site.xml**: facteur de réplication (**dfs.replication**) , NameNode secondaire (**dfs.namenode.secondary.http-address**), …

…….



Hadoop: Lecture d'un fichier HDFS

1. Le client (application HDFS) communique avec le NameNode pour demander la lecture d'un fichier (ou plusieurs, un répertoire par exemple)
2. Le NameNode envoie des métadonnées au client avec le détail du nombre de blocs, des identifiants et tailles des blocs, des emplacements des blocs et du nombre de réplication.
3. Le client communique avec les DataNodes où les blocs sont présents.
4. Le client commence à lire les blocs en parallèle à partir des DataNodes en se basant sur les métadonnées reçues du NameNode.
5. Pour améliorer les performances de lecture, l'emplacement de chaque bloc est choisi en fonction de sa distance par rapport au client: d'abord lecture des blocs dans le même DataNode, puis un autre DataNode dans le même rack, et enfin un autre DataNode dans un autre rack.
6. Une fois que le client reçoit tous les blocs, il les combine pour former un fichier.



Hadoop: Lecture d'un fichier HDFS (4/4)

❶ Le client appelle la méthode **open()** de l'objet **FileSystem** pour lire un fichier HDFS.

❷ Cet objet se connecte au **NameNode** à l'aide d'un appel **RPC** pour avoir les métadonnées du fichier à lire telles que les emplacements des blocs du fichier.

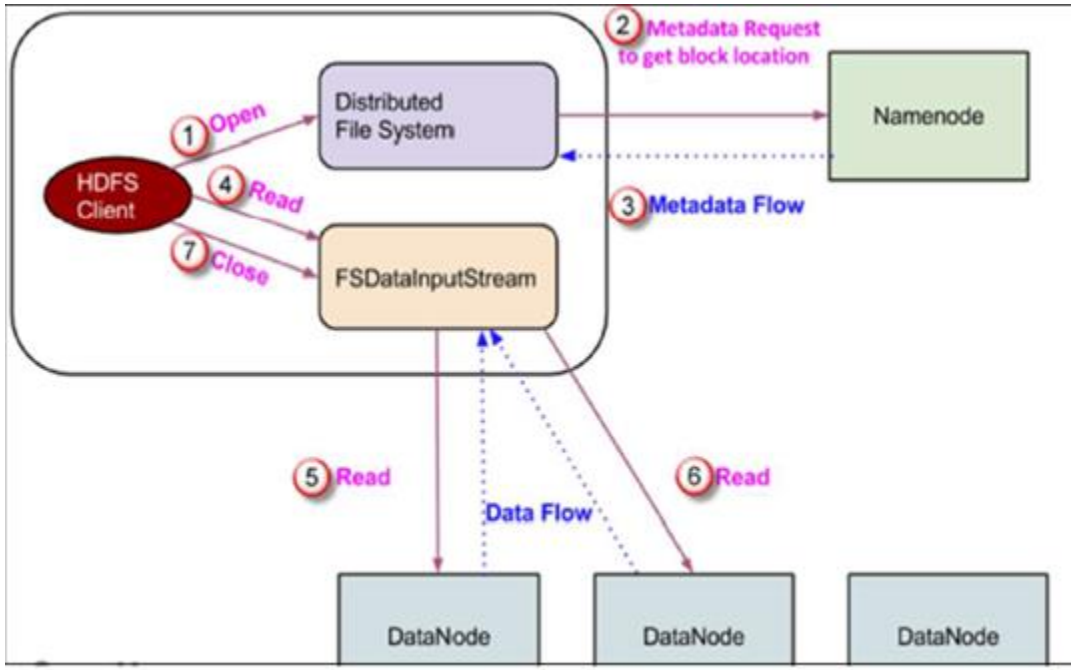
❸ Le NameNode envoie les métadonnées au client avec le détail du nombre de blocs, des identifiants et les tailles de blocs, des emplacements des blocs et du nombre de réplication.

❹ Le client appelle la méthode **read ()**, ce qui permet à un objet **DFSInputStream** d'établir une connexion avec un **DataNode** contenant un bloc du fichier

❺❻Le client lit ce bloc sous forme de flux via des appels répétitifs de **read()**. A la fin d'un bloc, **DFSInputStream** ferme la connexion avec le **DataNode**.

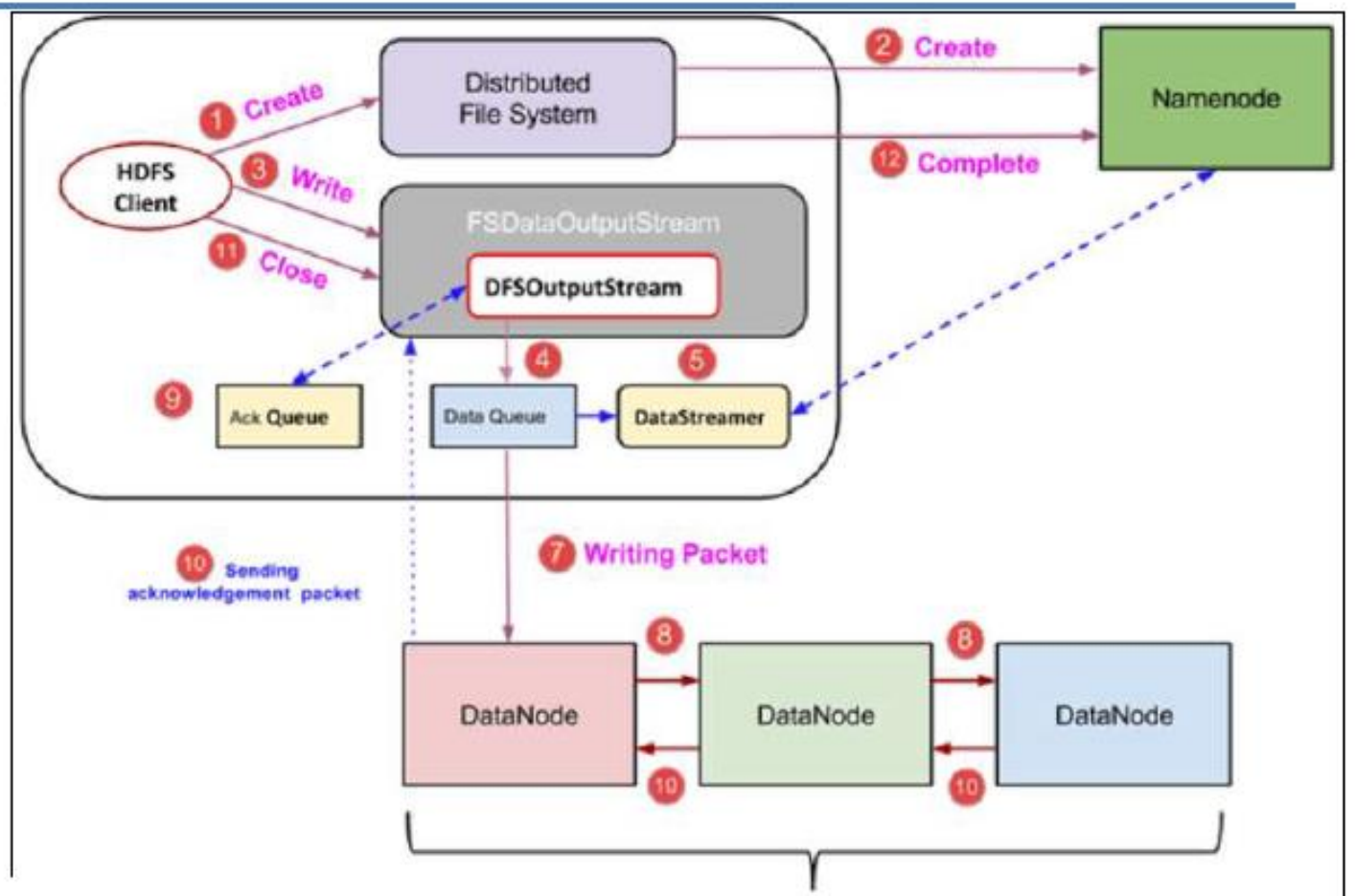
❼La méthode **close()** est appelée à la fin de la lecture de tous les blocs.

Une fois que le client reçoit tous les blocs, il les combine pour former un fichier.



Référence: https://www.guru99.com/learn-hdfs-a-beginners-guide.html

Hadoop: Ecriture d'un fichier dans HDFS (1/4)



Référence:
https://www.guru99.com/learn-hdfs-a-beginners-guide.html

Hadoop: Ecriture d'un fichier dans HDFS (2/4)

1. Le client appelle la méthode **create()** de l'objet **DistributedFileSystem** qui permet de de créer un nouveau fichier pour écriture.
2. Cet objet se connecte au **NameNode** à l'aide de **RPC** et lui demande la création d'un nouveau fichier. A ce stade on n'associe aucun bloc à ce fichier. Le **NameNode** vérifie s'il y a déjà un autre fichier de même nom et si le client à les droits nécessaires pour créer le fichier. Dans ce cas, une entrée (dans le FsImage) est créée pour le nouveau fichier. Dans le cas contraire, une exception **IOException** est soulevée.
3. Un objet de la classe **FSDataOutputStream** est retourné au client afin de l'utiliser pour écrire le contenu du fichier dans **HDFS** en invoquant la méthode **write()** (en boucle)
4. L'objet **FSDataOutputStream** utilise un autre objet **DFSOutputStream** qui gère la communication avec les **DataNodes** et le **NameNode**. **DFSOutputStream** crée à fur et à mesure des paquets (64k) avec les données écrites par le client. Ces paquets sont mis dans une file d'attente appelée **DataQueue**. **DFSOutputStream** décompose ainsi les fichiers à écrire en paquets.

Hadoop: Ecriture d'un fichier dans HDFS (3/4)

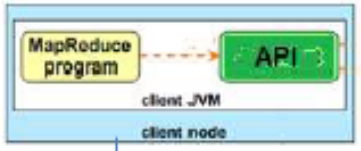
5. Le composant **DataStreamer** consomme les paquets de **DataQueue** et demande au **NameNode** l'allocation de nouveaux blocs. Le **NameNode** envoie les métadonnées au client avec le détail du nombre de blocs, des identifiants de blocs, des emplacements des blocs et du nombre de réplication. Ainsi les **DataNodes** à utiliser pour la réplication de chaque bloc seront identifiés.
6. Le processus de réplication d'un bloc commence par la création d'un pipeline à l'aide de **DataNodes** identifiés dans l'étape 5. Dans le schéma, le facteur de réplication est 3, il y a donc 3 **DataNodes** dans le pipeline.
7. Le **DataStreamer** envoie des paquets vers le premier **DataNode** du pipeline
8. Chaque **DataNode** d'un pipeline stocke le paquet reçu et le transmet au **DataNode** suivant du pipeline.
9. **DFSOutputStream** gère une autre file d'attente, "**Ack Queue**", pour stocker les paquets qui attendent un accusé de réception de **DataNodes**.

Hadoop: Ecriture d'un fichier dans HDFS (4/4)

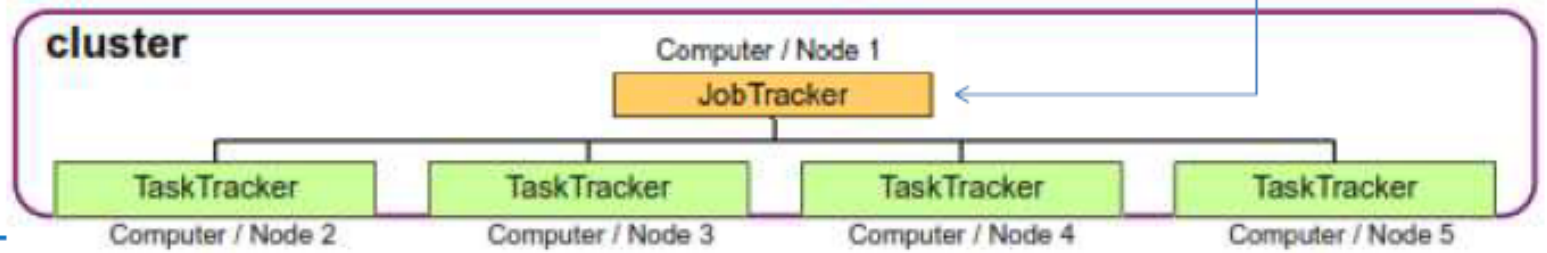
10. Lorsque l'accusé de réception d'un paquet dans la file d'attente est reçu de tous les nœuds de données du pipeline, il est supprimé de la file «Ack Queue». En cas d'échec de DataNode, les paquets de cette file d'attente sont utilisés pour relancer l'opération.
11. Le client termine l'écriture des données par l'appel de la méthode close (), ce qui entraîne l'envoie des paquets de données restants vers le pipeline, puis l'attente de l'accusé de réception.
12. Une fois l'accusé de réception final reçu, le NameNode est contacté pour lui indiquer que l'opération d'écriture de fichier est terminée.

MapReduce: JobTracker

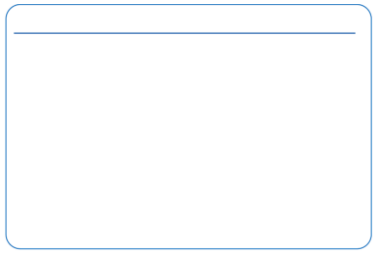
- Reçoit les **jobs MapReduce** envoyés par les clients (Applications)
- Communique avec le **NameNode** pour avoir les emplacements des données à traiter
- Passe les tâches **Map** et **Reduce** aux nœuds **TaskTrackers**
- Surveille les tâches et le statut des **TaskTrackers**
- Relance une tâche si elle échoue.
- Surveille l'état d'avancement des jobs et fournit des informations à ce sujet au applications clientes.
- Algorithme d'ordonnancement des jobs est par défaut FIFO.



➔ JobTracker fait: la *gestion des ressources*, l'*ordonnancement* et la *surveillance des tâches*



RÉFÉRENCE



- Cours Pr N. EL FADDOULI


FIN